

Relatório - Fase 3

Desenvolvimento de Software	—	Prof. Dr. Marcelo Soares Pimenta	—	INF01120
<u>Davi Santos</u>				<u>00599530</u>
<u>Gabriel Schons</u>				<u>00342020</u>
<u>Gabriel Copatti</u>				<u>00342808</u>
<u>Julio Augusto de Castilhos Borges</u>				<u>00590537</u>

Introdução

Dos critérios do checklist original, foram escolhidos:

- Código Duplicado: Pois a duplicação de lógica dificulta correções de bugs, o que causa modificações em múltiplos locais.
- Classe Grande: Facilita o acoplamento excessivo e reduz a coesão das classes.
- Método Longo: Torna a leitura e compreensão do código exaustivas, aumentando a chance de introdução de regressões.
- Inveja de Recursos: Revela acoplamento inadequado de controle onde uma classe manipula ativamente dados internos de outra.
- Obsessão por Tipos Primitivos: Prejudica a tipagem forte do Python e impede o uso de regras polimórficas.
- Switch Statements (ou if-else longos): Viola diretamente o Princípio Aberto-Fechado (OCP), exigindo modificações na lógica principal do parser para adicionar qualquer novo comando.

CrITÉRIOS DO CHECKLIST ORIGINAL

1. Classe Grande:

- src/core/gerenciador_midi.py #L10-L46

```
# código em ba67af3
class GerenciadorMidi(IGerenciador_arquivo):
    def __init__(self):
        self.__evento_midi = InterpretadorMidi()
        self.__arq_midi = MidiFile(type=1, ticks_per_beat=480)
        self.caminho = ""
    ...
    def processar_arquivo(self, vozes: list[Track]):
        #! Loop para parsing diretamente no gerenciador de arquivos?!
        for i, voz in enumerate(vozes):
            ...
            while j < len(texto):
                char = texto[j]
            # Lógica de interpretação misturada...
```

- **Descrição do Problema:** A classe GerenciadorMidi acumula a responsabilidade de gerenciar arquivos físicos de áudio no disco (criar, salvar, remover) juntamente com a lógica complexa de processamento de texto caractere por caractere (interpretação da sintaxe musical). Isso viola o Princípio de Responsabilidade Única (SRP). Se as regras de mapeamento de caracteres mudarem, a classe de gerenciamento de arquivos precisará ser refatorada, o que não faz sentido.
- **Sugestão de Refatoração:** Extrair a responsabilidade de **parsing** e interpretação para uma nova classe dedicada, como InterpretadorTexto ou ParserMusical, deixando a classe GerenciadorMidi focada exclusivamente no gerenciamento do arquivo MIDI físico usando a biblioteca mido.

```
# Proposta: Divisão em duas classes coesas
class ParserMusical:
    def __init__(self, interpretador: InterpretadorMidi):
        self.interpretador = interpretador

    def converter_texto_para_midi(self, vozes: list[Track], midi_file: MidiFile):
        # Transfere a lógica do loop 'while' caractere-a-caractere para cá
        ...
```

```
# A classe GerenciadorMidi passará a ser simplificada:
class GerenciadorMidi(IGerenciador_arquivo):
    def __init__(self, parser: ParserMusical):
        self.__parser = parser
        self.__arq_midi = MidiFile(type=1, ticks_per_beat=480)
        # Gerenciamento de arquivos apenas
```

2. Método Longo

- src/core/gerenciador_midi.py #L34-L94

```
# código em ba67af3
def processar_arquivo(self, vozes: list[Track]):
    for i, voz in enumerate(vozes):
        ...
        while j < len(texto):
            char = texto[j]
            if char in "ABCDEFGG" and ...: # Bemol
                ...
            if char in self.__evento_midi.notas_midi: # Nota
                ...
            elif char in 'abcdefgh': # Pausa
                ...
            elif char in '?.': # Oitava
                voz.oitava += 1
            ...
            j += 1
```

- **Descrição do Problema:** O método possui mais de 60 linhas e engloba o loop de tokenização, tratamento de bemol, controle de trinado/repetição, alteração de BPM, alteração de oitava e cálculo de volume. O fluxo de controle é denso, com aninhamentos, tornando a manutenção difícil e impedindo o teste unitário isolado de partes menores da lógica de tradução de notas.
- **Sugestão de Refatoração:** Quebrar o método em funções privadas auxiliares como _tratar_bemol, _processar_nota, _processar_comando_controle e _processar_repeticao_ou_pausa.

```
# Proposta de quebra do método longo:
def processar_arquivo(self, vozes: list[Track]):
    for i, voz in enumerate(vozes):
        track = self.criaTrack(track_name=f'voz {i}')
        self._inicializar_trilha(track, voz, i)
        self._processar_texto_da_voz(track, voz, i)

def _processar_texto_da_voz(self, track, voz, canal):
    j = 0
    texto = voz.texto_track
    ultima_nota = ""
    while j < len(texto):
        char, incremento = self._extrair_comando(texto, j)
        ultima_nota = self._executar_comando(char, track, voz, canal, ultima_nota)
        j += incremento
```

3. Inveja de Recursos

src/core/gerenciador_midi.py: #L34-L94 e src/mixer.py: #L43-L73)

- **Descrição do Problema:**
 1. Em gerenciador_midi.py, o processador de arquivo manipula diretamente os campos privados e estados de controle internos de Track (modifica voz.oitava, voz.volume) e de InterpretadorMidi (self.__evento_midi.bpm_global), em vez de delegar essas mutações para as próprias classes detentoras das informações.
 2. Em mixer.py, o método start interage diretamente com atributos visuais de componentes do Tkinter (campo.campo_texto.get(), campo.param_volume.get(), etc.) para criar as instâncias de Track, acoplando o domínio com a interface do usuário.
- **Sugestão de Refatoração:**
 - Delegar o controle de oitavas e alteração de volumes para o Track ou InterpretadorMidi por meio de métodos como voz.aumentar_oitava(), voz.duplicar_volume().
 - No Mixer, receber uma lista de estruturas limpas contendo os dados (DTO) ou extrair a leitura de dados na classe da GUI e injetá-los no Mixer desacoplados de widgets Tkinter.

```
# Em track.py:
def incrementar_oitava(self):
    self.oitava += 1
    ...

def decrementar_oitava(self):
    self.oitava -= 1
    ...

def duplicar_volume(self):
    self.volume *= 2
    ...
```

4. Obsessão por tipos primitivos

- **Localização:** GerenciadorMidi.processar_arquivo (gerenciador_midi.py: L48-92)

- **Descrição do Problema:**

O código usa caracteres de string isolados diretamente na lógica de decisão (char in 'abcde', char in 'V', char in '?') e representa notas musicais como Strings brutas ("A", "Eb"). Isso espalha “números mágicos” de caracteres por toda a aplicação e perde os benefícios de orientação a objetos.

Sugestão de Refatoração:

Definir constantes ou classes de comando (como ComandoAumentarOitava, ComandoTocarNota) ou utilizar dicionários que mapeiem caracteres para objetos que conhecem sua própria operação.

```
# Proposta de encapsulamento
class ComandoMusical(ABC):
    @abstractmethod
    pass

    def executar(self, track: MidiTrack, voz: Track, interpretador:
InterpretadorMidi):

    class AumentarOitava(ComandoMusical):
        def executar(self, track: MidiTrack, voz: Track, interpretador:
InterpretadorMidi):
            voz.incrementar_oitava()
```

5. Switch Statements

- **Localização:** src/core/gerenciador_midi.py - processar_arquivo() L60-86

- **Descrição do Problema:** O método possui 10+ branches if/elif para tratar cada tipo de caractere. Isso viola o Princípio Aberto-Fechado (OCP): adicionar novo comando exige modificar o método principal.

- **Sugestão de Refatoração:** Usar padrão Command ou Strategy: criar dicionario_dispatch = {'A': processar_nota, 'b': processar_pausa, ...} que pode ser estendido sem modificar o fluxo principal.

CrITÉRIOS NOVOS

1. Violação da Inversão de Dependência (DIP)

- **Descrição do Problema:** O módulo io_manager.py (Core) importa diretamente tkinter.filedialog (UI). Isso cria um acoplamento rígido entre o negócio e o framework gráfico, impedindo execução sem interface gráfica.
- **Sugestão de Refatoração:** A camada de UI deve coletar o caminho do arquivo e passá-lo para o Core. O IOManager deve lidar apenas com operações nativas de disco.

2. Getter com Efeitos Colaterais

- **Descrição do Problema:** O getter da propriedade delay em Track altera o estado do objeto ao ser chamado (self.__delay = 0). Getters devem ser idempotentes.
- **Sugestão de Refatoração:** Tornar o getter puramente consultivo e criar um método explícito como consumir_delay() para resetar o valor.

3. Abuso de Herança de Implementação

- **Descrição do Problema:** InterpretadorMidi herda de MIDIDConfig apenas para reuso de atributos. Conceitualmente, um interpretador não “é uma” configuração.
- **Sugestão de Refatoração:** Utilizar Composição sobre Herança. Injetar uma instância de MIDIDConfig no construtor do InterpretadorMidi.

4. Qualidade dos Testes Unitários

- **Descrição do Problema:** A suíte de testes está quebrada devido a chamadas a métodos inexistentes (ex: track.eh_atrasado()).
- **Sugestão de Refatoração:** Corrigir os testes existentes e adicionar cobertura para o núcleo de tradução do parser.

Pontos Fortes

1. **Arquitetura GUI Modular:** Divisão clara em subclasses no pacote src/ui.
2. **Máquina de Estados:** Uso de MixerState para controlar o ciclo de vida da aplicação.
3. **Interfaces de Domínio:** Uso de IGerenciador_arquivo como base para abstrações.
4. **Callbacks para Desacoplamento:** Coordenação entre UI e Mixer via funções de callback.

Conclusão

O projeto possui uma base sólida e modular, mas requer ajustes em princípios de design (SOLID) e na suíte de testes. As prioridades são: corrigir os testes quebrados, desacoplar o Core do framework gráfico e eliminar efeitos colaterais em métodos de leitura de estado.